MongoDB Basic Practice Task

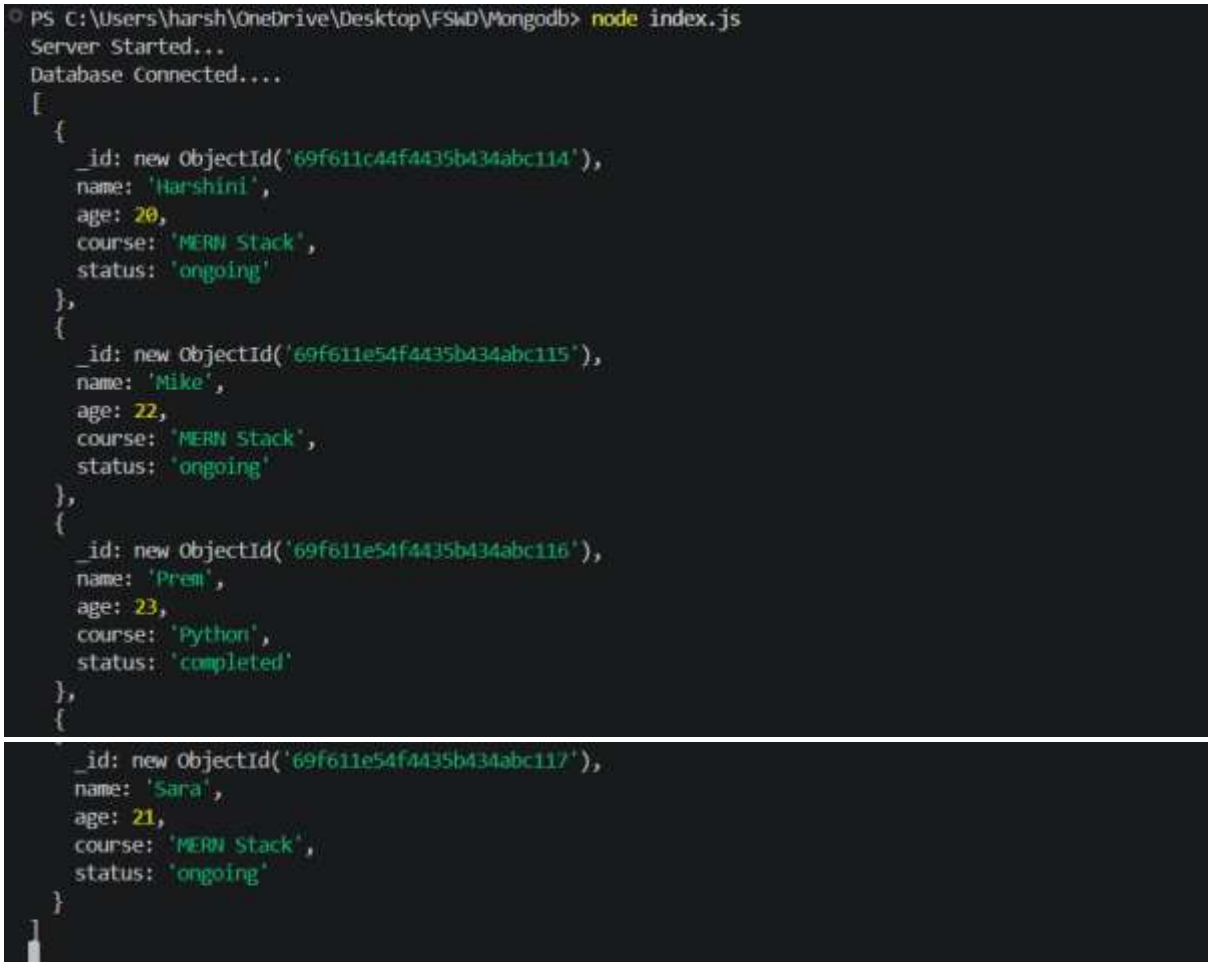
```
const express = require("express")
const mongoose = require("mongoose")

const app = express()

mongoose.connect("mongodb://127.0.0.1:27017/studentDB").
    then(function () {
        console.log("Database Connected....")
    })
    .catch(function () {
        console.log("Failed to connect with DB")
    })

//Create Model
const students = mongoose.model("students", {
    name:String,
    age:Number,
    course:String,
    status:String
},"students")
```

```
app.get("/students", function (req, res) {
    students.find().then(
        function(retdata)
        {
            console.log(retdata)
            res.send("Successfully Retrieved Data"+ retdata)
        }
    )
        .catch(function () {
            console.log("Error is retrieving Data")
        })
})

app.listen(3000, function () {
    console.log("Server Started...")
})
```

```
Microsoft Windows [Version 10.0.26100.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\harsh>mongosh
Current Mongosh Log ID: 69f611864f0435b434abc113
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:          8.2.7
Using Mongosh:          2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-02T08:10:04.804+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------
```

```
PS C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb> npm init -y
Wrote to C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb\package.json:

{
  "name": "mongodb",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1",
    "mongoose": "^9.6.1"
  },
  "devDependencies": {},
  "description": ""
}



PS C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb> npm install express

up to date, audited 83 packages in 2s

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
PS C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb> node index.js
Server Started...
Database Connected....
PS C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb> npm install mongoose

up to date, audited 83 packages in 2s

23 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```



1. Database & Collection Setup:

o Create a new database (e.g., studentDB)

o Create a collection (e.g., students)

2. Insert Operations:

o Insert one document into the students collection.

o Insert multiple documents with fields like name, age, course, status.

```
studentDB> db.students.insertOne({name: "Harshini", age: 20, course: "MERN Stack", status: "ongoing"})
{
  acknowledged: true,
  insertedId: ObjectId('69f611c44f4435b434abc114')
}
studentDB> db.students.insertMany([{ name: "Mike", age: 22, course: "MERN Stack", status: "ongoing" }, { name:
 "Prem", age: 23, course: "Python", status: "completed" }, { name: "Sara", age: 21, course: "MERN Stack", stat
us: "ongoing" }])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f611e54f4435b434abc115'),
    '1': ObjectId('69f611e54f4435b434abc116'),
    '2': ObjectId('69f611e54f4435b434abc117')
  }
}
```

3. Read Operations:

o Fetch all documents from the collection.

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc116'),
    name: 'Prem',
    age: 23,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

Successfully Retrieved Data [_id: new ObjectId('69f611c44f4435b434abc114'), name: 'Harshini', age: 20, course: 'MERN Stack', status: 'ongoing' }, { _id: new ObjectId('69f611e54f4435b434abc115'), name: 'Mike', age: 22, course: 'MERN Stack', status: 'ongoing' }, { _id: new ObjectId('69f611e54f4435b434abc116'), name: 'Prem', age: 23, course: 'Python', status: 'completed' }, { _id: new ObjectId('69f611e54f4435b434abc117'), name: 'Sara', age: 21, course: 'MERN Stack', status: 'ongoing' }

o Use filters to fetch specific data (e.g., students enrolled in “MERN Stack”).

```
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

4. Update Operations:

o Update a single document (e.g., change the status of a student to completed).

```
studentDB> db.students.updateOne({ name: "Mike" },  { $set: { status: "completed" } } )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB>
```

o Update multiple documents at once.

```
studentDB> db.students.updateMany( { course: "MERN Stack" },  { $set: { status: "completed" } } )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 2,
  upsertedCount: 0
}
studentDB>
```

5. Query Operators:

o Practice using $gt, $lt, $in, $and, $or, $exists in different queries.

```
studentDB> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc116'),
    name: 'Prem',
    age: 23,
    course: 'Python',
    status: 'completed'
  }
]
studentDB>
```

```
studentDB> db.students.find({ age: { $lt: 23 } })

[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
```

```
studentDB> db.students.find({ course: { $in: ["MERN Stack", "Python"] } })
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc116'),
    name: 'Prem',
    age: 23,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
```

```
studentDB> db.students.find({$and: [{ age: { $gt: 20 } }, { status: "ongoing" }]})

studentDB>
```

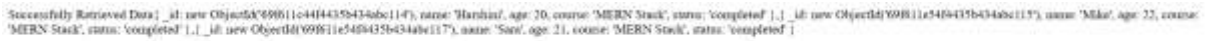
```
studentDB> db.students.find({$or: [{ course: "Python" }, { age: { $lt: 21 } }]})
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc116'),
    name: 'Prem',
    age: 23,
    course: 'Python',
    status: 'completed'
  }
]
```

```
studentDB> db.students.find({ age: { $exists: true } })
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc116'),
    name: 'Prem',
    age: 23,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
```

5. Delete Operations:

o Delete one document based on a condition.

```
studentDB> db.students.deleteOne({ name: "Prem" })
{ acknowledged: true, deletedCount: 1 }
studentDB>  db.students.find()
[
  {
    _id: ObjectId('69f611c44f4435b434abc114'),
    name: 'Harshini',
    age: 20,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc115'),
    name: 'Mike',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('69f611e54f4435b434abc117'),
    name: 'Sara',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  }
]
studentDB>
```



o Delete all documents in the collection (for practice purpose only).

```
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
studentDB> db.students.find()

studentDB>
```

Successfully Retrieved Data

7. Use Case:

o Create a small use case like a Library System or E-Commerce product listing and apply insert, update, and search operations.

Outcome:

```
const express = require("express")
const mongoose = require("mongoose")

const app = express()

mongoose.connect("mongodb://127.0.0.1:27017/shopDB").
    then(function () {
        console.log("Database Connected....")
    })
    .catch(function () {
        console.log("Failed to connect with DB")
    })

//Create Model
const products = mongoose.model("products", {
    name:String,
    price:Number,
    category:String
},"products")
```

```
app.get("/products", function (req, res) {
    products.find().then(
        function(retdata)
        {
            console.log(retdata)
            res.send("Successfully Retrieved Data"+ retdata)
        }
    )
        .catch(function () {
            console.log("Error is retrieving Data")
        })
})

app.listen(3000, function () {
    console.log("Server Started...")
})
```

```
studentDB> use shopDB
switched to db shopDB
shopDB> db.products.insertMany([
| { name: "Laptop", price: 50000, category: "Electronics" }, { name: "Phone", price: 20000, category: "Electronics" }
, {name: "Shirt", price: 1000, category: "Clothing" } ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f6e4134f4435b434abc118'),
    '1': ObjectId('69f6e4134f4435b434abc119'),
    '2': ObjectId('69f6e4134f4435b434abc11a')
  }
}
```

```
shopDB> db.products.find()
[
  {
    _id: ObjectId('69f6e4134f4435b434abc118'),
    name: 'Laptop',
    price: 50000,
    category: 'Electronics'
  },
  {
    _id: ObjectId('69f6e4134f4435b434abc119'),
    name: 'Phone',
    price: 20000,
    category: 'Electronics'
  },
  {
    _id: ObjectId('69f6e4134f4435b434abc11a'),
    name: 'Shirt',
    price: 1000,
    category: 'Clothing'
  }
]
shopDB>
```

```
PS C:\Users\harsh\OneDrive\Desktop\FSWD\Mongodb> node index.js
Server Started...
Database Connected....
[
  {
    _id: new ObjectId('69f6e4134f4435b434abc118'),
    name: 'Laptop',
    price: 50000,
    category: 'Electronics'
  },
  {
    _id: new ObjectId('69f6e4134f4435b434abc119'),
    name: 'Phone',
    price: 20000,
    category: 'Electronics'
  },
  {
    _id: new ObjectId('69f6e4134f4435b434abc11a'),
    name: 'Shirt',
    price: 1000,
    category: 'Clothing'
  }
]
```

Successfully Retrieved Data [_id: new ObjectId('69f6e4134f4435b434abc118'), name: 'Laptop', price: 50000, category: 'Electronics' }, { _id: new ObjectId('69f6e4134f4435b434abc119'), name: 'Phone', price: 20000, category: 'Electronics' }, { _id: new ObjectId('69f6e4134f4435b434abc11a'), name: 'Shirt', price: 1000, category: 'Clothing' }

```
shopDB> db.products.find({ price: { $gt: 15000 } })
[
  {
    _id: ObjectId('69f6e4134f4435b434abc118'),
    name: 'Laptop',
    price: 50000,
    category: 'Electronics'
  },
  {
    _id: ObjectId('69f6e4134f4435b434abc119'),
    name: 'Phone',
    price: 20000,
    category: 'Electronics'
  }
]
```

```
shopDB> db.products.updateOne({ name: "Phone" }, { $set: { price: 18000 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
shopDB> db.products.find()
[
  {
    _id: ObjectId('69f6e4134f4435b434abc118'),
    name: 'Laptop',
    price: 50000,
    category: 'Electronics'
  },
  {
    _id: ObjectId('69f6e4134f4435b434abc119'),
    name: 'Phone',
    price: 18000,
    category: 'Electronics'
  },
  {
    _id: ObjectId('69f6e4134f4435b434abc11a'),
    name: 'Shirt',
    price: 1000,
    category: 'Clothing'
  }
]
shopDB>
```

Successfully Retrieved Data[{ _id: new ObjectId('69f6e4134f4435b434abc118'), name: 'Laptop', price: 50000, category: 'Electronics' }, { _id: new ObjectId('69f6e4134f4435b434abc119'), name: 'Phone', price: 18000, category: 'Electronics' }, { _id: new ObjectId('69f6e4134f4435b434abc11a'), name: 'Shirt', price: 1000, category: 'Clothing' }]